ACS-2947-002/003
Assignment 1
Due by Tuesday, February 4, 11:59 PM

Instructions

- Submit your `.java` files (together in a `Assign1.zip` file) via Nexus.
- Include your name and student number as a comment in every file.
- Document the classes using Javadoc notation.
- Include comments as needed and use exception handling where necessary.

PART A – LinkedLists (55 marks)

In this simple turn-based multiplayer game, each player takes a turn rolling two dice. After each roll their score is increased by the result (the score of the two dice added together) and the next player will have their turn. The following rules also applies in the game:

- If a player's roll includes a 2, increase their score by double the roll amount.
- If a player's roll results in a total of 7, their score is decreased by 7 points, unless the roll included a 2 which is treated as above. Note that scores do not go below 0.
- If a player rolls a pair of 6s, their score is resets to 0 and the turn taking order is reversed.
- If the player rolls a pair of 1s, they are eliminated from the game.

This continues until a winner is declared: either someone reaches 100 points or there is only one player left in the game.

1. Program the generic `CircularDoublyLinkedList` class. See appendix for CDLL representation.
   - Use the circularly and doubly linked list classes from our notes as a basis.
   - Start with the DLL and think of how it can work circularly.
     - Sentinels are not required, only reference to the last node.
     - `Node` will have references to the next and previous nodes.
     - `addBetween` and `remove` private utilities will be used, all adding and removing will work like doubly.
   - Your CircularlyDoubleLinkedList class will support the following public methods:

     `size()` Returns the number of elements in the list.
     `isEmpty()` Returns true if the list is empty, and false otherwise.
     `first()` Returns (but does not remove) the first element in the list.
     `last()` Returns (but does not remove) the last element in the list.
     `addFirst(e)` Adds a new element to the front of the list.
     `addLast(e)` Adds a new element to the end of the list.
     `removeFirst()` Removes and returns the first element of the list.
     `removeLast()` Removes and returns the last element of the list.
     `rotate()` Advances to the next element in the list based on direction
     `reverse()` Changes the direction

2. Create the following classes:
   a) `Die` stores the number of faces and the face value of a die. Include a method to roll a die (returning a random value between 1 and the number of faces on the die).
   b) `Dice` stores two instances of `Die`.
      - Add a default constructor to create pair of dice where the number of sides on each is 6.
      - Add a constructor that allows the number of sides in the dice to be specified.
      - Create a `roll()` method that rolls the two dice and returns an `ArrayList` containing the face value of the two dice.
   c) `Player` stores the player's name and their score information. Include any methods required to retrieve or update the score.
   d) `Game` class that simulates a game between a number of players.
      Note: most of the work will be done here. `Assign1PartA_Driver` should have only minimal amount of code: e.g., declare/create a Game instance, add players, initiate game play.
      - Fields
        - a `CircularDoublyLinkedList` of players
        - a `Dice`
        - any other field required to manage the game
      - Upon instantiation, set up the list of players (use 4 players for your demo/sample data) and create the dice. Begin the game with the first player in the list. Each player will have their turn when positioned at the head of the list.
      - After each player's turn, the face values of the dice and the current score of the player is announced.
      - The final output should include the winner, the final scores of the remaining players and an indication of the players that were eliminated, if any.

3. Write a driver class called `Assign1PartA_Driver` that creates a list of four players of your choice and invokes a simulation of the game.

Suggestions:

- Use `java.util.Random` to generate random numbers
- Override the `toString` method in the necessary classes

Sample Output

```
The Dice Game!

Alvin rolled 4 and 3. Unlucky 7, score is now 0.
Simon rolled 3 and 6. Score is now 9.
Theodore rolled 6 and 2. Lucky 2, score is now 16.
David rolled 5 and 2. Lucky 2, score is now 14.
Alvin rolled 3 and 6. Score is now 9.
Simon rolled 1 and 2. Lucky 2, score is now 15.
Theodore rolled 6 and 5. Score is now 27.
David rolled 5 and 6. Score is now 25.
```

```
Alvin rolled 6 and 3. Score is now 18.
Simon rolled 5 and 3. Score is now 23.
Theodore rolled 5 and 6. Score is now 38.
David rolled 5 and 3. Score is now 33.
Alvin rolled 6 and 6. Double 6, score is now 0. Reverse direction.
David rolled 2 and 5. Lucky 2, score is now 47.
Theodore rolled 1 and 4. Score is now 43.
Simon rolled 3 and 5. Score is now 31.
Alvin rolled 4 and 3. Unlucky 7, score is now 0.
David rolled 4 and 4. Score is now 55.
Theodore rolled 3 and 2. Lucky 2, score is now 53.
Simon rolled 2 and 2. Lucky 2, score is now 39.
Alvin rolled 3 and 4. Unlucky 7, score is now 0.
David rolled 3 and 1. Score is now 59.
Theodore rolled 1 and 3. Score is now 57.
Simon rolled 6 and 4. Score is now 49.
Alvin rolled 4 and 2. Lucky 2, score is now 12.
David rolled 3 and 6. Score is now 68.
Theodore rolled 2 and 2. Lucky 2, score is now 65.
Simon rolled 4 and 4. Score is now 57.
Alvin rolled 2 and 6. Lucky 2, score is now 28.
David rolled 1 and 1. Snake eyes! Player eliminated :(
Theodore rolled 3 and 2. Lucky 2, score is now 75.
Simon rolled 5 and 5. Score is now 67.
Alvin rolled 4 and 2. Lucky 2, score is now 40.
Theodore rolled 6 and 5. Score is now 86.
Simon rolled 4 and 6. Score is now 77.
Alvin rolled 2 and 3. Lucky 2, score is now 50.
Theodore rolled 4 and 2. Lucky 2, score is now 98.
Simon rolled 3 and 4. Unlucky 7, score is now 70.
Alvin rolled 4 and 3. Unlucky 7, score is now 43.
Theodore rolled 3 and 6. Score is now 107.
Theodore won the game! :)

Final scores:
Simon: 70, Theodore: 107, Alvin: 43
Eliminated: [David]
```

PART B - Queues (25 marks)

When a share of common stock of some company is sold, the *capital gain* (or, sometimes, loss) is the difference between the share's selling price and the price originally paid to buy it. This rule is easy to understand for a single share, but if we sell multiple shares of stock bought over a long period of time, then we must identify the shares actually being sold. A standard accounting principle for identifying which shares of a stock were sold in such a case is to use a FIFO protocol—*the shares sold are the ones that have been held the longest.*

For example, suppose we buy 100 shares at $20 each on day 1, 20 shares at $24 on day 2, 200 shares at $40 on day 3, and then sell 150 shares on day 4 at $30 each. Applying the FIFO protocol means that of the 150 shares sold, 100 were bought on day 1, 20 were bought on day 2, and 30 were bought on day 3. The capital gain in this case would therefore be 100 * 10 + 20 * 6 + 30 * (−10), or $820.

You will develop a Java program that takes as input from the keyboard a sequence of transactions to buy and sell shares. Given this input sequence, the output of selling shares should show the total capital gain (or loss), using the FIFO protocol to identify shares.

1. Program the generic `ArrayQueue` from our class notes, that implements the given `Queue` interface using an array.
2. Create an `Assign1PartB_Driver` class that will take input from the keyboard. Your program should execute like the example given below, as well as any other valid combination of purchases and sales that the user may make.

3. Have the queue hold values that represent the buying price of each stock. Each buy transaction would queue the buying price. E.g., (for the first sample transaction) $20 is enqueued 100 times.

4. Create an `Account` class that handles and records the stock transactions. The class should contain:
    - A `Queue` variable i.e., `shares`
    - Methods:
        - `buyShares(…)` adds the number of shares and share price to the queue
        - `sellShares(…)` removes the specified number of shares from the queue calculates the capital gain/loss for the transaction
        - `getTotalValue()` returns the running total value of shares
        - `getTotalShares()` returns the running total number of shares

5. Include the following in your driver
    - Methods to display your menu and summary
    - Input validation and exception handling

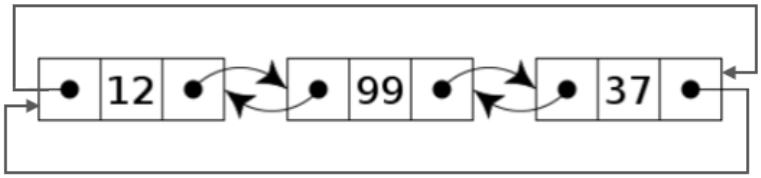
Sample Output

```
Make a selection:
1. Buy shares
2. Sell shares
3. Quit
1

Enter number of shares to buy: 100
Enter purchase price per share: 20

Total shares: 100
Total shares value: $2000.0

----------------------------------------

Make a selection:
1. Buy shares
2. Sell shares
3. Quit
1

Enter number of shares to buy: 20
Enter purchase price per share: 24
```

```
Total shares: 120
Total shares value: $2480.0

----------------------------------------

Make a selection:
1. Buy shares
2. Sell shares
3. Quit
1

Enter number of shares to buy: 200
Enter purchase price per share: 36

Total shares: 320
Total shares value: $9680.0

----------------------------------------

Make a selection:
1. Buy shares
2. Sell shares
3. Quit
2

Enter number of shares to sell: 150
Enter the selling price per share: 30

Capital gain: $940.0

Total shares: 170
Total shares value: $6120.0

----------------------------------------

Make a selection:
1. Buy shares
2. Sell shares
3. Quit
3

*** End of Program ***
```

Submission

Submit your **`Assign1.zip`** file that include all the assignment files (`CircularDoublyLinkedList.java`, `Die.java`, `Dice.java`, `Game.java`, `Player.java`, `Assign1PartA_Driver.java`, `Queue.java`, `ArrayQueue.java`, `Account.java`, `Assign1PartB_Driver.java`) and any other java file you used via **Nexus**.

Appendix A (CDLL Representation)

The structure of a CDLL storing Integer types is shown below.